

Scheduling Algorithms

Dispatcher Vs Scheduler:

Dispatcher: it does the context switch by saving the current process' registers in the PCB (process control block) and loads the saved registers of the next process into the CPU so it can resume execution. Registers (= small storage inside the CPU (program counter, stack pointer, general-purpose registers, etc.))

Scheduler: Decides which process is to be run next and for how long. Changes the state of each process (blocked, waiting, running).

Criteria for Scheduling Processes

When the OS decides which process runs next, it tries to make things "good" based on:

1. CPU Utilization / Load → Keep the CPU busy as much as possible.
2. Response Time → How fast a process starts responding (important for interactive programs).
3. Turnaround Time → Total time a process takes from start to finish.
4. Waiting Time → How long a process waits in the ready queue.
5. Real-Time Behaviour → Meeting deadlines for time-sensitive tasks.
6. Throughput → How many processes finish in a certain time. (how many processes does the cpu finish per second) more throughput better performance
7. Overhead → Keep scheduling efficient; don't waste CPU on switching too much.
8. Fairness → Make sure every process gets a chance to run

Main Criteria from Processes' Perspective

Arrival Time (AT): The time that takes a process from the moment system starts into the moment its in the ready queue

Completion Time (CT): The time it took the process to finish from the start of the system until the process is executed and terminated $CT = AT + TAT$

CPU time, Burst Time (BT): The time a process actually spends on the CPU being ran, excluding waiting time and everything else.

Waiting Time (WT): the time the process had to wait in the ready queue before being picked by the CPU

Turn Around Time ($TAT = CT - AT$): total time of the process from being ready in the queue into the moment it is finished. $TAT = WT + BT$

Non-Preemptive Scheduling (Simple Version):

- A process keeps running on the CPU until it either:
 1. Finishes completely, or
 2. Voluntarily gives up the CPU (like waiting for input)
- Key point: The OS cannot force a running process to stop.
- Problem: If a process runs for a long time, other processes have to wait → CPU can be “stuck”
- Old examples: Windows 3.x, MacOS 8/9, Windows 95/98/ME

Effect of Non-Preemptive on Execution Order:

Lets assume we have 2 process PA and PB and both have the AT of 0 they both started at the exact time our system booted.

PA has CPU time of 27 seconds, PB has CPU of 3 seconds.

We will assume 2 scenarios.

We start with PA the PB.

Our WT, PA = 0 and PB = 27

Our TAT, PA = 27, PB = 30

Avg TAT = 28.5, avg WT = 13.5

We Start with PB then PA

Our WT, PA = 3, PB = 0

Our TAT, PA = 30, PB = 3

Aavg TAT = 16.5, avg WT = 1.5

This shows that for a non-preemptive OS its better and more efficient to start with processes that ends faster. This takes down the waiting time and the turnaround time.

Preemptive Scheduling:

A running process can be stopped or pause at anytime even if its not finished or it did not give up the cpu (waiting for input or action)

Pros: this supports multi-tasking, and is more time efficient than FCFS, shorter waiting times.

Cons: Context switching is more work for OS, process with low priority may starve. Multiple process running at the same time might cause an issue.

Overhead = extra work or time the OS spends doing administrative tasks instead of actual computation.

Scheduling Methods (Selection)

Non-Preemptive Scheduling:

First Come, First Served (FCFS)

Shortest Job First (SJF) •

Priority Scheduling (non-preemptive)

Earliest Deadline First (EDF) (non-preemptive)

Preemptive Scheduling:

Shortest Remaining Time First (SRTF)

Round Robin (RR) • Priority Scheduling (preemptive)

Earliest Deadline First (EDF) (preemptive)

Multilevel Scheduling

First Come First Served (non-preemptive):

It uses first in first out queue to assign a process to a cpu core.

The average waiting time and the average turnaround time might be too high, it's the easiest and simplest OS algorithm method.

It's a fair algorithm that makes sure all processes execute no process starving.

FCFS is great for situations where jobs can wait their turn and don't need fast response, like printing, payroll processing, or large data computations "Good for batch processing".

Used for Linux non-preemptive real time processes.

Arrival Time of Proc A = 0, B = 2ms, C = 5ms

In this example since we use FCFS we will start with A since it has arrival time of 0 then B then C.

The Answer for the example is 2 for the avg WT and 7.3 for the avg TAT

Round Robin (RR) - Time Slicing Algorithm (Preemptive):

It uses also a FIFO queue for tasks but in this case it give each process a certain time interval that varies, after this time interval if the process is completed the process lets the CPU go, if its not completed it is moved back to the tail of the queue.

Like a circle queue.

Its for time sharing systems, it prevents starvation.

Pros: simple and behaves like FCFS, more responsive and time sharing, no starving.

Cons: too much overhead time, short process may wait longer than it should.

Answer to the question:

Avg WT: to be studied in more details

Avg TAT: to be studied in more details

Priority based Scheduling:

Each process has a priority value assigned to it by the OS or the User we have couple of priority types. Processes with the highest priority gets the running state. If 2 processes have the same priority they go into a FCFS state. Using this you might risk starvation of processes with low priority.

Types of Priorities

1. Fixed / Static Priority
 - Priority is assigned once and doesn't change.
 - Example: A system process may always have higher priority than a user process.
2. Dynamic Priority
 - Priority changes over time depending on behavior (like waiting too long).
 - Example: Multilevel Feedback Scheduling adjusts priority so that long-waiting processes eventually get CPU.

Priority-based scheduling is a concept, not a single rule.

Depending on the OS design, it can be implemented as:

1. Non-preemptive → a running process finishes before switching, even if a higher-priority process arrives. Then picks the process with the current highest priority and execute it until its finished.
2. Preemptive → a running process can be interrupted if a higher-priority (or sometimes same-priority) process arrives.

The higher the priority value the higher the priority.

Avg TAT and AVG WT non = 17.75, 9.75 ms, assuming B has the highest priority

Avg TAT and AVG WT pre = ,assuming A has the highest priority

Earliest Deadline First:

Processes that are the closest to be finished always gets the highest priority the scheduler changes the priorities dynamically whenever a new process enters the ready queue. Then it selects the process with the EDL to be assigned to the CPU or a process terminates also changes the priority.

We use this algorithm in a non preemptive way with batch processing

And use the preemptive ones with linux RTOS real time operating system

Issues with common scheduling methods

1. Some methods (like SJF, SRTF, LJF, LRTF) are hard to use in real life because:
 - You often don't know CPU burst times in advance
2. Even if you estimate from past runs, it's just a model → not perfect
3. All methods (Priority, FCFS, RR, EDF) have trade-offs:
 - You can't optimize all criteria (waiting time, turnaround, fairness, etc.) at once

Most simple scheduling methods are impractical alone, so OSs use multilevel scheduling to handle real-world trade-offs.

Multilevel Feedback Scheduling:

A preemptive scheduling method.

The processes are queued in multilevel and each internal queue and outer queue uses round robin method meaning that the process inside the second queue cannot be picked after the highest priority queue is finished.

How processes move between queues in MLFQ

1. Voluntary wait (e.g., I/O)
 - If a process pauses on its own (like waiting for input), it stays in the same queue
 - Reason: it's not hogging the CPU, so it keeps its current priority
2. Uses full time quantum
 - If a process uses all its allowed CPU time without pausing, it is moved to a lower-priority queue
 - Reason: it's CPU-bound → give higher-priority CPU time to other processes

Advantages of Multilevel Feedback Scheduling (MLFQ)

1. **Preference for newer processes**
 - New processes usually start in the **highest-priority queue** → get CPU quickly
 - Makes the system **more responsive** for recent tasks
2. **I/O-bound processes are favored**
 - Processes that **frequently wait for input/output** don't use up their full time quantum
 - They **stay in higher-priority queues**, so they get CPU faster when ready
 - Improves **overall system efficiency**, because CPU-bound processes don't block short I/O tasks

Many modern OS uses variants of this method:

- Linux (up to kernel 2.4) • Mac OS X • FreeBSD, NetBSD • MS Windows NT